

StatsAI — Code Review & Fix Guide

Date: 2026-07-04

Scope: Web (Next.js 14 App Router) + Prisma/PostgreSQL backend, cron scripts, and the Expo/React Native mobile app.

Method: Five parallel reviewers (auth/security, cron & push pipeline, core data libs, web API routes & pages, mobile app). Every HIGH finding below was re-verified against the actual code before inclusion; low-confidence or style-only items were dropped.

Severity legend: ● HIGH = wrong data, auth/security, or silent breakage affecting real users · ● MEDIUM = incorrect behavior with a workaround or narrow trigger · ● LOW = edge cases, polish, hardening.

Note: several HIGH items (H3, H5, and the push-binding H4) originate in code changed during recent sessions — they are latent right now only because the league is off-season and there is no live traffic. They will surface the moment the 2026/27 season kicks off.

● HIGH

H1 · Rotated refresh tokens are usable as full web sessions (reuse-detection bypass)

Where: `src/lib/auth.ts:87–118` (`getCurrentUser` , `getRequestUser`) vs `src/app/api/mobile/v1/auth/refresh/route.ts:75–78` .

Problem: On refresh, the old Session row is only stamped `replacedAt` and kept until its ~60-day `expiresAt` . The session lookups match by `tokenHash` and check only `expiresAt` + `isActive` — never `replacedAt` . Reuse detection only runs inside `/refresh` .

Failure scenario: An attacker who captured a refresh token the victim has since rotated does NOT call `/refresh` (which would trip reuse detection). Instead they set it as the `hbs_session` cookie → `getCurrentUser` matches the still-unexpired replaced row → authenticated as the victim for the token's full original lifetime. Rotation/reuse-detection is effectively defeated.

Fix: Reject replaced sessions in both lookups.

```
// getCurrentUser + getRequestUser (cookie branch)
const session = await prisma.session.findUnique({
  where: { tokenHash: sha256(rawToken) },
  include: { user: true },
});
if (!session || session.replacedAt || session.expiresAt < new Date() ||
!session.user.isActive) return null;
```

Optionally add a `kind: 'web' | 'mobile'` column so a mobile refresh token can never be presented as a web cookie at all.

H2 · Mobile users are logged out on every server deploy (refresh cleared on 5xx)

Where: `mobile/lib/apiClient.ts:39-42`.

Problem: `performRefresh` does `if (!res.ok) { await clearRefreshToken(); return null; }` — it treats *any* non-OK, including a transient 502/503 during `pm2 restart`, as an invalid token and wipes the keychain (which also clears the stored user).

Failure scenario: Access token expires → a request 401s → refresh fires exactly during a deploy or rate-limit window → 502 → refresh token deleted → user silently, permanently logged out (next launch boots logged-out).

Fix: Only clear on genuine auth failure.

```
if (!res.ok) {  
  if (res.status === 401 || res.status === 403) await clearRefreshToken();  
  return null; // leave the token intact on 5xx / rate-limit / transient errors  
}
```

H3 · Live tab and home "live" section go empty once games are live (Hebrew/English mismatch)

Where: `src/lib/mobile-api.ts:615` (home) and `src/app/api/mobile/v1/live/route.ts` (live tab filter added for the World Cup exclusion) vs `src/lib/home-live.ts:100,581`.

Problem: `countryLabel` is passed through `translateLiveText`, and `LIVE_TRANSLATIONS['Israel'] = 'ישראל'`, so an Israeli match's `countryLabel` is always `'ישראל'`. Both filters compare against the English `'Israel'`:

- home: `.filter(s => s.countryLabel === 'Israel')` → never matches.
- live tab: `if (snapshot.countryLabel !== 'Israel') continue;` → drops **everything**.

Failure scenario: Any live Israeli match → dropped by both filters → the mobile home "live" block and the entire Live tab render empty (while `summary.liveCount` still reports matches live). Latent now only because it's off-season.

Fix: Filter on the raw (untranslated) country, not the display label. Expose it on the snapshot and compare it:

```
// home-live.ts mapSnapshotToHomepage – add the raw country alongside the label  
country: rawLeague.country || null, // e.g. "Israel" (untranslated)  
countryLabel, // "ישראל" (display only)
```

```
// mobile-api.ts:615 and live/route.ts  
.filter(s => s.country === 'Israel') // home  
if (snapshot.country !== 'Israel') continue; // live tab
```

H4 · Devices stop receiving personalized pushes after any app restart (token unbinds to userId=null)

Where: `mobile/app/_layout.tsx:46-48` + `src/app/api/mobile/v1/notifications/register/route.ts`.

Problem: `registerForPushNotifications()` runs on every cold start, but the in-memory access token is null then (only login or a 401→refresh populates it), and the register endpoint is auth-optional so it never 401s. The backend upsert does `update: { userId: user?.id ?? null }` → an unauthenticated re-register **overwrites the binding to null**. Push targeting resolves recipients via the user relation, so the device stops getting goal/result/reminder pushes. (This is exactly the "guest token" observed earlier.)

Fix: Don't register (or don't send a re-bind that can null the user) until authenticated.

```
// _layout.tsx – register on login, and on cold start only if a session exists
useEffect(() => { if (user?.id) registerForPushNotifications(); }, [user?.id]);
```

And/or make the backend never downgrade an existing binding: `userId: user?.id ?? existing.userId ?? null`.

H5 · 0-0 scores written onto unplayed games (reminders × matches interaction)

Where: `scripts/notify-reminders.js:96-100` + `scripts/notify-matches.js:112-118,158-159`.

Problem: The reminder cron upserts a `GameNotificationState` row (defaults 0-0, not-final) ~1h before kickoff. `notify-matches` then treats any game with state + not-final + not-COMPLETED as "dropped off the live feed" → fetches `/fixtures?ids=` → gets NS with `goals: null` → `?? 0` → writes `{homeScore:0, awayScore:0, status:'SCHEDULED'}` onto the `Game` row.

Failure scenario: Every scheduled game shows a fake 0-0 result for the hour before kickoff; consumers that key on non-null scores display/count it wrongly; ~20 wasted API calls per game per pre-kickoff hour.

Fix: Only finalize games that plausibly started, and never write scores for scheduled games.

```
// notify-matches.js – droppedIds filter: require kickoff in the past
.filter((g) => !live.has(g.apiFootballId) && states.get(g.id)
  && !states.get(g.id).notifiedFinal
  && g.status !== 'COMPLETED'
  && new Date(g.dateTime).getTime() <= Date.now()) // ← add
```

```
// the game.update at line 158-159: skip when the fixture is still scheduled
if (cur.status !== 'SCHEDULED' && (g.homeScore !== cur.home || g.awayScore !== cur.away ||
g.status !== cur.status)) {
  await prisma.game.update({ where: { id: g.id }, data: { homeScore: cur.home, awayScore:
cur.away, status: cur.status } }).catch(() => null);
}
```

Related (same file): make `notifiedFinal` non-regressing — `notifiedFinal: state?.notifiedFinal || cur.status === 'COMPLETED'` — so an FT→ONGOING API flap can't cause a duplicate final push.

H6 · Standings / stats / mobile default to the not-yet-started 2026/27 season

Where: `src/app/standings/page.tsx:146`, `src/app/statistics/page.tsx:85`, `src/app/api/mobile/v1/standings/route.ts:18-23`, `src/app/api/mobile/v1/stats/route.ts:35-48`; root cause `src/lib/home-live.ts:369` (`getCurrentSeasonStartYear` returns 2026 from July, but the league kicks off in late August).

Problem: These default to `year <= getCurrentSeasonStartYear() = 2026`. Season 2026 now has friendlies + a zeroed pre-season standings row, so the default table is empty or friendly-derived instead of the completed 2025/26 table. (The `/games` fix — "newest season WITH games" — does not help, because 2026 *has* games.)

Fix: Introduce a shared "current display season" helper that returns the newest season with **completed league play**, and use it for defaults:

```
// pick the newest season that has at least one COMPLETED league (383/382) game
export async function getDefaultDisplaySeasonId() {
  const s = await prisma.season.findFirst({
    where: { games: { some: { status: 'COMPLETED', competition: { apiFootballId: { in: [383, 382] } } } } },
    orderBy: { year: 'desc' },
    select: { id: true },
  });
  return s?.id ?? null;
}
```

Until the new league starts this returns 2025/26; once real league games are played it advances. Apply in all four call sites (and consider it for the AI `getStandings` / `getTeamCardSummary` default too — see M8).

H7 · Standing rows are clobbered across competitions (silent data loss)

Where: `prisma/schema.prisma:604` (`Standing @@unique([seasonId, teamId])`, `competitionId nullable`) + `src/app/api/admin/fetch/route.ts:1975-2028`.

Problem: The unique key allows only one Standing row per team per season, but the fetch upserts on (seasonId, teamId) while setting competitionId. Fetching a *second* competition's standings for teams that also play in the league overwrites the league row and flips its competitionId.

Failure scenario: Admin fetches Toto Cup (385) group standings, or a UEFA table for an Israeli club → those teams' Ligat Ha'al rows are overwritten and vanish from every league-table view until 383 standings are re-fetched.

Fix (schema change): make standings unique per competition.

```
model Standing {
  // ...
  competitionId String // make required (backfill existing nulls first)
  @@unique([seasonId, teamId, competitionId])
}
```

Then change the upsert where to the 3-part key. Requires prisma db push + a data backfill for any legacy null competitionId rows.

H8 · Mobile game-rating feature is 100% broken and hides the failure

Where: mobile/design-system/GameRatingSheet.tsx:60,72,78-79.

Problem: apiClient prepends /api/mobile/v1, but the sheet calls apiClient.get('/api/games/\${id}/rate') → requests /api/mobile/v1/api/games/:id/rate, which doesn't exist (404). The catch { /* ignore */ } swallows it, so the UI looks saved but nothing persists and averages are always empty. Line 72 also routes to /auth/login (correct route is /login).

Fix: Call the correct path and surface errors. Either add a mobile route src/app/api/mobile/v1/games/[id]/rate/route.ts, or call the existing web endpoint with an absolute base:

```
// use a path WITHOUT the v1 prefix (add an apiClient.rawPost, or fetch config.apiUrl +
'/api/games/:id/rate')
await apiClient.post(`/games/${gameId}/rate`, { ratings }); // once a mobile/v1 rate route
exists
// and fix the guest redirect:
router.push('/login');
```

Also add the game-participation + array-size guards from L-rate below on whichever endpoint mobile calls.

M1 · AI chatbot reports wrong standings/champion. `src/lib/ai-tools.ts:286–311` returns raw `s.points` (ignores `pointsAdjustment`, e.g. Ironi Tiberias –8, Hapoel TA –2) and orders by stored pre-playoff `position`, so it names the regular-season leader "champion" mid-playoff and quotes points that contradict the order. **Fix:** select `pointsAdjustment`, compute `adjustedPoints = points + pointsAdjustment`, sort by it (mirror `src/lib/standings.ts:21–28`); soften the "Position 1 is the champion" tool description during playoffs.

M2 · Sofascore stats panel mislabels every metric under RTL. `mobile/design-system/SofascoreMatchStatsPanel.tsx:73–90` hardcodes `flexDirection: 'row'` with the away block first; under `isRTL` it flips so away renders on the visual right and home on the left — opposite of the panel's own legend and the rest of the app. **Fix:** use `rtlRow()` with the home block first (`match games/[id].tsx`).

M3 · Mobile "predictions" and "head-to-head" show stale/oldest games. `src/lib/mobile-api.ts:362–375, 376–389` lack the web's `game: { status: 'SCHEDULED', dateTime: { gte: now } }` filter and order by `dateTime asc / gameId asc`, so they surface the season's earliest (already-played) fixtures. **Fix:** add the SCHEDULED-future filter (predictions) and the SCHEDULED-future / last-7-days filter (H2H), matching `src/app/page.tsx:320, 325–338`.

M4 · notify-matches can double-send goals/finals. `scripts/notify-matches.js:102–160` has no run lock and the `fetch` has no timeout, so a >3-min run overlaps the next 3-min run; both read the same `GameNotificationState` before either upserts → duplicate pushes. **Fix:** add a `flock/advisory-lock` guard and an `AbortController` `fetch` timeout (e.g. 60s); move the state upsert before the send, or make sending idempotent per (game, event).

M5 · Telegram text over-capture pollutes news pushes and the feed. `src/lib/telegram.ts:75–81` matches lazily to the first `</div></div>`, which for messages with reaction/view blocks lands past the text — verified live, extracted text ends with "... 76 views edited 12:28 ♥ 2". **Fix:** anchor the text-div regex to the message-text container only, or strip trailing `tgme_widget_message_footer /reaction/ views` fragments before use.

M6 · Playoff standings glitches. `src/lib/standings-from-games.ts:123–133` returns only championship+relegation rows once one game of each is complete, so teams whose group is still `null` disappear; and mobile maps `position: row.displayPosition` (per-group 1..N) → duplicate position numbers (`src/lib/mobile-api.ts:571`). **Fix:** include not-yet-grouped teams (fall through to a combined list) and map the continuous `position`, not `displayPosition`, on mobile.

M7 · Coach win-chart re-merges distinct coaches. `src/lib/coach-timeline.ts:243–295` re-derives keys via `normalizeKey(displayName)` (initial + surname), collapsing two different coaches who share those (e.g. two managers curated as separate `Coach` rows) into one series — undoing the canonicalization the per-season timeline just fixed. **Fix:** carry the canonical `coachId` from `buildCoachTimelineBySeason` into the chart key instead of re-normalizing the display name.

M8 · AI defaults to empty future/artifact seasons. `getTeamCardSummary` (`ai-tools.ts:341`) and the mobile player payload (`src/app/api/mobile/v1/players/[id]/route.ts:126`) use `season.findFirst({ orderBy: { year: 'desc' } })` with no `year <= getCurrentSeasonStartYear()` guard, so they resolve to 2026/27 (or a kept future artifact) → "no data"/all-clear answers. **Fix:** apply the same current-season guard used elsewhere (and prefer the H6 display-season helper).

M9 · AI getPlayerEvents truncates careers at 50 events. `ai-tools.ts:194-211` uses `take: 50` newest-first with no marker, while the system prompt (`ai-providers.ts:47`) tells the model it returns the full career → wrong career totals. **Fix:** raise/remove the cap for career queries, or return a `truncated` flag and total count so the model doesn't miscount.

M10 · Social-login routes are unthrottled. `src/app/api/auth/{google,apple}` and `src/app/api/mobile/v1/auth/{google,apple}` don't call `checkRateLimit`, unlike `login/refresh/reset` — each request triggers an outbound IdP/JWKS verification + DB work. **Fix:** add a per-IP `checkRateLimit` to all four.

M11 · Password reset link logged; email is a no-op stub. `src/lib/email.ts:18-27` `console.log`s the full message body (containing the raw reset URL) and returns `delivered:false`; no provider is wired. **Fix:** wire a real email provider; never log the token/link (log only "reset requested for <userId>").

🟡 LOW

- **SideMenu RTL:** drawer docks on the physical LEFT in RTL (`SideMenu.tsx:39,65` assume logical left/right, but `swapLeftAndRightInRTL(false)` makes them physical) — should use `right:0` like `standings.tsx:157`. And the "משחקים" item routes to `/games`, which has **no route** (`app/games/index.tsx` doesn't exist) → Unmatched-Route dead end. Add a games index screen or remove the item.
 - **Team header anchoring** (`mobile/app/teams/[id].tsx:65`) uses `alignItems:'flex-end'` where the analogous player header uses `'flex-start'` — one anchors the name to the wrong side under RTL.
 - **Player "דקות" shows hours** (`mobile/app/players/[id].tsx:170`): `${Math.round(minutes/60)}h` under a label that says minutes.
 - **Favorite-team card** (`mobile/app/(tabs)/index.tsx:69`) uses `flex:1` on a Text inside an RTL row — the exact pattern documented as collapsing to blank in `SideMenu.tsx:114`. Switch to natural-width `flexShrink:1` (verify on device).
 - **notify-news secret in query string** (`src/app/api/cron/notify-news/route.ts:51`) lands in access logs — prefer the `x-cron-secret` header only.
 - **500 on bad input:** `?year=abc` on mobile `standings/stats` (`parseInt` → NaN → Prisma throw) and `players/sidelined` PUT/DELETE on unknown id / bad date — validate and return 400/404.
 - `/api/games/[id]/rate` accepts an unbounded `ratings` array and players who didn't play in the game, non-transactionally, feeding public averages — cap length, verify participation, wrap in a transaction.
 - **Events PUT** (`src/app/api/events/route.ts:146,176`) skips the type allowlist that POST enforces — apply the same allowlist.
 - **Refresh idempotency window** (`refresh-cache.ts`, checked before reuse detection) lets a replayed token return the cached tokens for 30s — acceptable tradeoff, but worth documenting; tighten if H1's `replacedAt` check isn't enough.
-

✓ Verified clean

- **Admin authz** — every mutating `/api/admin/**` and non-admin mutating route checks `role === 'ADMIN'`; `deleteUserAccount` uses a Serializable transaction for the last-admin guard (no TOCTOU).
- **JWT** pins `algorithms: ['HS256']` (no alg-confusion); **bcrypt** cost 12; token comparisons are hashed DB lookups.
- **apiClient** singleflight refresh + `_retried` guard + `/auth/` exclusion + post-refresh token re-read are correct.
- **AuthContext** session ordering, logout/delete cache clearing (`queryClient.clear()` + `persister.removeClient()`), and Google/Apple cancel paths are sound.
- **teamBadge** initials handle null/empty/all-prefix/single-word/symbol names without crashing.
- Import scripts are idempotent (upsert by stable keys); `buildEventApiFootballId` is collision-free (<1000 events/fixture) and safely nulls past int4 max.
- React list keys and stat divisions (win%, ratings, goal-timing) are id-based and zero-guarded.

Recommended fix order

1. **H2, H4** — silent auth/push breakage hitting real users now (small, mobile-only, ship via OTA).
2. **H3, H6** — wrong/empty data the moment the season starts (backend; deploy together).
3. **H1** — security (refresh-token bypass; one-line guard in two lookups).
4. **H5** — stop writing 0-0 to unplayed games (cron script; no deploy needed beyond `git pull`).
5. **H8** — mobile rating path (OTA).
6. **H7** — needs a schema change + backfill; schedule deliberately.
7. **MEDIUM** batch (M1–M11) as a follow-up pass; **LOW** as time permits.

H1/H2/H3/H4/H5/H8 are all small and can be batched into one release. H6 wants a shared season helper; H7 needs a migration.